

09 — Pipeline reproductible & data contracts

Objectif (data engineering). Formaliser un **contrat de données** (schéma, types, bornes, valeurs autorisées) validé automatiquement avec **pandera**, et montrer l'intérêt d'un pipeline reproductible avec **exports** standardisés (CSV / PDF / IPYNB). C'est la garantie qualité d'un entrepôt décisionnel : toute donnée non conforme est rejetée avant d'alimenter les indicateurs.

On opère sur le jeu réel constitué au notebook 01.

```
[1] from pathlib import Path

import pandera.pandas as pa

from lib import load_works, set_seeds

set_seeds()
df = load_works()
print(f"Jeu à valider : {len(df):,} lignes, {df.shape[1]} colonnes")
```

```
Jeu à valider : 4,000 lignes, 14 colonnes
```

1. Définition du contrat de données

Le schéma décrit les attentes : types, nullabilité, bornes et domaines de valeurs. C'est la spécification exécutable de ce qu'est une donnée « valide ».

```
[2] schema = pa.DataFrameSchema(
    {
        "id": pa.Column(str, nullable=False, unique=True),
        "year": pa.Column(int, pa.Check.in_range(2010, 2026)),
        "cited_by_count": pa.Column(int, pa.Check.ge(0)),
        "is_oa": pa.Column(bool),
        "oa_status": pa.Column(
            str, pa.Check.isin(["gold", "green", "hybrid", "bronze", "closed",
                                ]),
        ),
        "type": pa.Column(str, nullable=True),
        "n_authors": pa.Column(int, pa.Check.ge(0)),
        "abstract_len": pa.Column(int, pa.Check.ge(0)),
    },
    strict=False,
    coerce=True,
)
print("Contrat défini :", list(schema.columns))
```

```
Contrat défini : ['id', 'year', 'cited_by_count', 'is_oa', 'oa_status', 'type',
                  'n_authors', 'abstract_len']
```

2. Validation du jeu réel

La validation *lazy* collecte toutes les violations en une passe plutôt que de s'arrêter à la première.

```
[3] try:
    validated = schema.validate(df, lazy=True)
    print(f"✓ Validation réussie : {len(validated):,} lignes conformes au contrat")
except pa.errors.SchemaErrors as exc:
    print("Violations détectées :")
    print(exc.failure_cases[["column", "check", "failure_case"]].head())
```

```
✓ Validation réussie : 4,000 lignes conformes au contrat.
```

3. Détection d'une donnée non conforme

On injecte volontairement une anomalie (année impossible) pour montrer que le contrat la **rejette** — exactement ce qui protège l'entrepôt en production.

```
[4] corrupt = df.copy()
corrupt.loc[corrupt.index[0], "year"] = 1850
try:
    schema.validate(corrupt, lazy=True)
    print("Aucune anomalie détectée (inattendu).")
except pa.errors.SchemaErrors as exc:
    cases = exc.failure_cases[["column", "check", "failure_case"]]
    print(f"Anomalie correctement rejetée : {len(cases)} cas")
    print(cases.head())
```

```
Anomalie correctement rejetée : 1 cas
  column      check failure_case
0  year  in_range(2010, 2026)      1850
```

4. Exports standardisés (CSV / PDF / IPYNB)

Un livrable d'analyse doit être exportable. On produit un agrégat propre en CSV (réutilisable dans un tableur ou un autre outil). Le notebook lui-même est exportable en **IPYNB** (téléchargement direct) et en **PDF** (impression) depuis l'interface du dashboard.

```
[5] export_dir = Path("data/exports")
export_dir.mkdir(parents=True, exist_ok=True)

by_domain = (
    df.dropna(subset=["domain"])
    .groupby("domain")
    .agg(publications=("id", "size"), citations_moy=("cited_by_count", "mean"),
    .round({"citations_moy": 1, "taux_oa": 3})
    .sort_values("publications", ascending=False)
)
csv_path = export_dir / "production_par_domaine.csv"
by_domain.to_csv(csv_path, encoding="utf-8")
print(f"Export CSV écrit : {csv_path} ({csv_path.stat().st_size} octets)")
by_domain
```

```
Export CSV écrit : data/exports/production_par_domaine.csv (166 octets)
```

	publications	citations_moy	taux_oa
domain			
Physical Sciences	2294	16.6	0.746
Social Sciences	697	3.2	0.479
Health Sciences	586	16.5	0.611
Life Sciences	402	16.8	0.774

Lecture. Le contrat de données rend la qualité **exécutable et reproductible** : à chaque rafraîchissement, toute notice non conforme est détectée avant d'atteindre les indicateurs. Couplé aux exports standardisés (CSV / PDF / IPYNB), cela constitue la base d'un pipeline industrialisable, auditable et prêt à alimenter un SID.